

NextMatchdayBanner & Match Events

Feature specification & build prompt — extracted from the T9L.me football-league codebase (Next.js + Prisma/Postgres).

Read this first. Everything below is **soccer / futsal-specific**. The data model and rendering logic bake in assumptions that do *not* hold for other sports such as pickleball, tennis, or volleyball. Sport-coupling points are flagged inline with `SPORT-COUPLED` and summarized at the end under "*Adapting to other sports*." Treat the soccer rules as a reference implementation, not a portable one.

1. NextMatchdayBanner

Reference file: `src/components/NextMatchdayBanner.tsx`

A swipeable card carousel on the homepage that shows one matchday at a time, with navigation between adjacent matchdays and a live countdown to kickoff.

Props

```
interface NextMatchdayBannerProps {
  matchdays: Matchday[];
  selectedMatchdayId: string;
  onMatchdayChange: (id: string) => void;
  teams: Team[];
  goals: Goal[]; // soccer-shaped result data (see §2)
  lockToSelected?: boolean; // prevents auto-defaulting on session hydration
  leagueSlug?: string; // for canonical share URLs
}
```

Behaviour

- **"Next matchday" determination:** the first *unplayed* matchday where the signed-in user's team is *not* sitting out — `md.sittingOutTeamId !== session.teamId && md.matches[0].homeGoals === null`. "Unplayed" is signalled by `homeGoals === null`.
- **Auto-default:** a `useEffect` pre-selects the user's next playable matchday once the session finishes loading, unless `lockToSelected` is set (disabled on matchday detail pages so a deep link stays put).
- **Navigation:** left/right chevrons with disabled edge states, touch + mouse-drag swipe (50px threshold), dot indicators for quick-jump, and a tracked animation direction (`'left' | 'right'`).
- **Eyebrow label** switches between "YOUR NEXT MATCHDAY" / "MATCHDAY DETAILS" / "MATCHDAY RESULTS" based on an `isUserNextMatchday` flag.
- Delegates the timer to `<MatchdayCountdown>` and the body (fixtures + scorers) to `<MatchdayCard>`.

Countdown

Reference file: `src/components/MatchdayCountdown.tsx` — note the hard-coded match length.

```
const MATCH_DURATION_MS = 33 * 60 * 1000; // league-standard match length
const FALLBACK_MATCHDAY_DURATION_MS = 3 * 60 * 60 * 1000; // 3-hour fallback
```

- Computes the matchday's start/end window from `Match.endedAt` ; falls back to *last kickoff + 33 min*, then *first kickoff + 3 h*.
- Renders an animated "Live" state between start and end, a `Xd Xh Xm` countdown before start, and hides entirely after end.
- Ticks every second via `setInterval` . All times handled in JST.

⚠ **Sport-coupled.** `SPORT-COUPLED` The 33-minute match length, the "sitting-out team" concept, and the single-fixture-per-team assumption (`matches[0]`) are football-league constructs. A pickleball event (multiple short games, round-robin, or bracket play) does not fit a `matchday → matches[]` model with one fixture per team.

2. Match Events

Reference: `prisma/schema.prisma` , `src/lib/matchScore.ts` , `src/lib/dbToPublicLeagueData.ts`

Data model

```
model MatchEvent {
  id          String    @id @default(cuid())
  matchId     String
  kind        EventKind @default(GOAL) // enum currently has ONLY { GOAL }
  goalType    GoalType? // OPEN_PLAY | SET_PIECE | PENALTY | OWN_GOAL
  scorerId    String?   // null when isGuestScorer
  assisterId  String?
  minute      Int?      // nullable for after-the-fact entry
  beneficiaryTeamId String? // explicit, for cross-team / guest scorers
  isGuestScorer Boolean @default(false)
  isGuestAssister Boolean @default(false)
  createdById String?   // audit: admin vs player self-report
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

enum EventKind { GOAL } // only goals are modelled
enum GoalType { OPEN_PLAY SET_PIECE PENALTY OWN_GOAL }
```

Score model

- `Match.homeScore` / `awayScore` are **derived caches**, recomputed from events by `recomputeMatchScore()` on every event mutation.
- `Match.scoreOverride` (free text, e.g. `"3-0 (forfeit)"`) takes display precedence for forfeits / abandoned games.
- **Own goals:** the scorer belongs to the conceding team; `goalType=OWN_GOAL` flips the beneficiary during recompute.
- **Guests:** `scorerId=NULL`, `isGuestScorer=TRUE` + an explicit `beneficiaryTeamId` for off-roster players.

Public shape

Derived in `src/lib/dbToPublicLeagueData.ts`; consumed by the banner/card.

```
interface Goal {
  id; matchId; matchdayId;
  scoringTeamId; concedingTeamId;
  scorer: string;           // player name or "Guest"
  assister: string | null;
  minute?: number | null;
  goalType?: 'OPEN_PLAY' | 'SET_PIECE' | 'PENALTY' | 'OWN_GOAL' | null;
}
```

CRUD & reporting paths

- **Admin:** `adminCreateMatchEvent` / `adminUpdateMatchEvent` / `adminDeleteMatchEvent`. Validates admin session, match-in-league, beneficiary is one of the two match teams, scorer/assister are active league members (XOR with the guest flag), minute clamped 0–200. Each mutation recomputes the score inside a transaction.
- **Player self-report:** `submitOwnMatchEvent` — any linked player, opens at earliest kickoff, auto-approved with an audit trail, requires `leagueSlug` for multi-league disambiguation.

Existing sport configuration (already in the schema)

The system already branches **soccer vs futsal** — but both share the same goal-scoring event model:

```
enum BallType { SOCCER FUTSAL }
// plus: GoalSize, ThrowInType, GoalKickType, offsideRule,
//       backpassRule, playerFormat, per-ballType position vocab
//       (12 soccer positions / 4 futsal positions)
```

⚠ Sport-coupled — the central issue for other sports. **SPORT-COUPLED**

- The **only event kind is GOAL**, carrying `goalType`, `scorer`, `assister`, `minute` — the soccer/futsal scoring grammar.
- **Scoring is "one event = one goal."** Pickleball, tennis, and volleyball score in rallies/points within games within sets — a flat list of goal events recomputed to an integer cannot model points-in-games-in-sets, side-outs, deuce, or rally-vs-side-out scoring.
- **No concept of sets/games, serve, or per-game winner.** A best-of-3-to-11 result has nowhere to live here.
- `OWN_GOAL`, `PENALTY`, `SET_PIECE`, `assister`, and `minute` are meaningless outside football.

3. Adapting to other sports (e.g. pickleball)

For soccer/futsal the current stack — `MatchEvent(kind=GOAL)` + cached `homeScore/awayScore` + `Goal[]` public shape + 33-minute countdown — is a complete, working spec. For a different sport the implementation should **diverge, not reuse** it:

Concern	Soccer (reference)	Other sports — change to
Result model	Flat <code>MatchEvent</code> list, kind=GOAL	Points → games → sets structure; serve/side-out tracking; per-game scores
Public summary	<code>Goal[]</code> (scorer/assist/minute)	Sport-specific result shape, or generic <code>{games: [...], winner}</code>
Countdown	<code>MATCH_DURATION_MS = 33min</code> constant	Sport-aware / per-event duration config
Banner "next" logic	One fixture per team, "sitting-out team"	Round-robin / bracket-aware fixture resolution
Branching	Implicit soccer; <code>BallType</code> for futsal	Explicit sport discriminator the banner, countdown, card & event forms all branch on

Recommendation: introduce a sport discriminator (extend `BallType` or add a `sport` field) and branch the banner, countdown, card, and event forms on it — rather than overloading `EventKind` / `GoalType` with non-goal sports. Keep the soccer event grammar isolated behind that discriminator so each sport owns its own result model.

Source repo: t9l-website (Tennozu 9-Aside League). Next.js App Router · Prisma + Postgres (Neon) · Upstash Redis · NextAuth.
File/line references are illustrative anchors for an LLM build prompt, not guaranteed line numbers.